AI ASSISTED CODING LAB

ASSIGNMENT 10.2

ENROLLMENT NO :2503A51L01

BATCH NO: 19

NAME:B.RAJA GOPAL

TASK1

TASK1 DESCRIPTION:-**AI-Assisted Code Review (Basic Errors)**

- Write python program as shown below.
- Use an AI assistant to review and suggest corrections

```
def calcFact(n):
  result=1
  x=0
  for i in range(1,n):
      result=result*i
  return result

def main():
      num = 5
      FACT = calcFact(num)
      print("the factorial of",num,"is",FACT)
      t=10
      if FACT>10:
             print("BIG Number")
      else:
       print("small number")

main()
```

CODE:-

Error Code:-

t1.py > calcFact > n

```python
def calcFact(n):
    result=1
    x=0
    for i in range(1,n):
        result=result*i
    return result

def main():
    num = 5
    FACT = calcFact(num)
    print("the factorial of",num,"is",FACT)
    t=10
    if FACT>10:
        print("BIG Number")
    else:
        print("small number")

main()

```

OUTPUT:-

```
PS C:\Users\khaja\OneDrive\Pictures\Screenshots\cyc\New folder\ai_10> & C:/Users/khaja/anaconda3/python.exe
w folder/ai_10/t1.py"
the factorial of 5 is 24
BIG Number
PS C:\Users\khaja\OneDrive\Pictures\Screenshots\cyc\New folder\ai_10>
```

Correct Code:-

```python
def calcFact(n):
    # Initialize result to 1 (factorial of 0 is 1)
    result = 1
    # Loop from 1 to n (inclusive) to calculate factorial
    for i in range(1, n + 1):
        result *= i
    return result

def main():
    num = 5
    # Call the factorial function
    fact = calcFact(num)
    print("The factorial of", num, "is", fact)
    # Check if factorial is greater than 10
    if fact > 10:
        print("BIG Number")
    else:
        print("Small Number")

# Only run main if this file is executed directly
if __name__ == "__main__":
    main()
```

OUTPUT:-

```
PS C:\Users\khaja\OneDrive\Pictures\Screenshots\cyc\New folder\ai_10> & C:/Users/khaja/anaconda3/python.exe
w folder/ai_10/t1_1.py"
The factorial of 5 is 120
BIG Number
```

OBSERVATION:-

In this task, I wrote a Python program that contained basic syntax or logical errors. I then used an AI assistant to review the code and suggest corrections. The AI successfully identified the errors and provided a corrected version of the code, along with explanations. This helped me understand common mistakes and how to fix them efficiently.

TASK2

TASK2 DESCRIPTION:-**Automatic Inline Comments**

- Write the Python code for Fibonacci as shown below and execute.
- Ask AI to improve variable names, add comments, and apply PEP8 formatting (cleaned up).
- Students evaluate which suggestions improve readability most.

```
def f1(xX):
    a=0
    b=1
    c=2
    Zz=[a,b]
    while c<=xX:
        d=a+b
        Zz.append(d)
        a=b
        b=d
        c=c+1
    return Zz

def m():
      NN=10
      ans=f1(NN)
      print("fib series till",NN,":",ans)

m()
```

CODE:-

Given Code:-

t2.py > f1

```
1   def f1(xx):
2       a=0
3       b=1
4       c=2
5       Zz=[a,b]
6       while c<=xx:
7           d=a+b
8           Zz.append(d)
9           a=b
10          b=d
11          c=c+1
12      return Zz
13
14  def m():
15      NN=10
16      ans=f1(NN)
17      print("fib series till",NN,":",ans)
18
19  m()
20
```

Output:-

```
PS C:\Users\khaja\OneDrive\Pictures\Screenshots\cyc\New folder\ai_10> & C:/Users/khaja/anaconda3/python.exe
w folder/ai_10/t2.py"
fib series till 10 : [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
PS C:\Users\khaja\OneDrive\Pictures\Screenshots\cyc\New folder\ai_10>
```

AI Generated Code:-

t2_1.py > ...

```python
def generate_fibonacci_series(limit):
    """
    Generate Fibonacci series up to a given limit.
    Args:
        limit (int): Number of terms in the Fibonacci series.
    Returns:
        list: Fibonacci series as a list.
    """
    first = 0
    second = 1
    count = 2
    fibonacci_series = [first, second]

    while count <= limit:
        next_term = first + second
        fibonacci_series.append(next_term)
        first = second
        second = next_term
        count += 1

    return fibonacci_series

def main():
    num_terms = 10
    series = generate_fibonacci_series(num_terms)
    print(f"Fibonacci series till {num_terms}:", series)

if __name__ == "__main__":
    main()
```

Output:-

```
PS C:\Users\khaja\OneDrive\Pictures\Screenshots\cyc\New folder\ai_10> & C:/Users/khaja/anaconda3/python.exe
w folder/ai_10/t2_1.py"
Fibonacci series till 10: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55]
PS C:\Users\khaja\OneDrive\Pictures\Screenshots\cyc\New folder\ai_10>
```

OBSERVATION:-

For this task, I implemented a Python program to generate Fibonacci numbers. After writing the initial code, I asked the AI assistant to improve variable names, add comments, and format the code according to PEP8 guidelines. The AI-generated version was clearer and easier to understand, especially for beginners.

TASK3

TASK3 DESCRIPTION:-

- Write a Python script with 3–4 functions (e.g., calculator: add, subtract, multiply, divide).
- Incorporate manual **docstring** in code with NumPy Style
- Use AI assistance to generate a module-level docstring + individual function docstrings.
- Compare the AI-generated docstring with your manually written one

CODE:-

calculator.py > add

```
"""
calculator.py

A simple calculator module providing basic arithmetic operations: addition, subtraction, multiplication, and division.
Each function accepts two numeric arguments and returns the result of the operation.

Examples
--------
>>> add(2, 3)
5
>>> subtract(5, 2)
3
>>> multiply(3, 4)
12
>>> divide(10, 2)
5.0
"""

def add(a, b):
    """
    Add two numbers.

    Parameters
    ----------
    a : float or int
        First number to add.
    b : float or int
        Second number to add.

    Returns
    -------
    float or int
        The sum of a and b.

    Examples
    --------
    >>> add(2, 3)
    5
    """
    return a + b
```

```python
def divide(a, b):
    """
    Divide one number by another.

    Parameters
    ----------
    a : float or int
        Numerator.
    b : float or int
        Denominator.

    Returns
    -------
    float
        The result of a divided by b.

    Raises
    ------
    ZeroDivisionError
        If b is zero.

    Examples
    --------
    >>> divide(10, 2)
    5.0
    """
    if b == 0:
        raise ZeroDivisionError("Cannot divide by zero.")
    return a / b

def main():
    print("Addition: add(2, 3) =", add(2, 3))
    print("Subtraction: subtract(5, 2) =", subtract(5, 2))
    print("Multiplication: multiply(3, 4) =", multiply(3, 4))
    print("Division: divide(10, 2) =", divide(10, 2))
    # Example for division by zero
    try:
        print("Division: divide(5, 0) =", divide(5, 0))
    except ZeroDivisionError as e:
        print("Division: divide(5, 0) raised an error:", e)

if __name__ == "__main__":
    main()
```

OUTPUT:-

```
PS C:\Users\khaja\OneDrive\Pictures\Screenshots\cyc\New folder\ai_10> & C:/Users/khaja/anaconda3/python.exe "c:/Users/khaja/OneDrive/Pictures/Screenshots
folder/ai_10/calculator.py"
Addition: add(2, 3) = 5
Subtraction: subtract(5, 2) = 3
Multiplication: multiply(3, 4) = 12
Division: divide(10, 2) = 5.0
Division: divide(5, 0) raised an error: Cannot divide by zero.
PS C:\Users\khaja\OneDrive\Pictures\Screenshots\cyc\New folder\ai_10>
```

OBSERVATION:-

In this task, I created a Python script containing multiple functions such as add, subtract, multiply, and divide. I manually wrote docstrings using the NumPy style format to describe each function’s purpose and parameters. Then, I asked the AI to generate its own module-level and function-level docstrings.